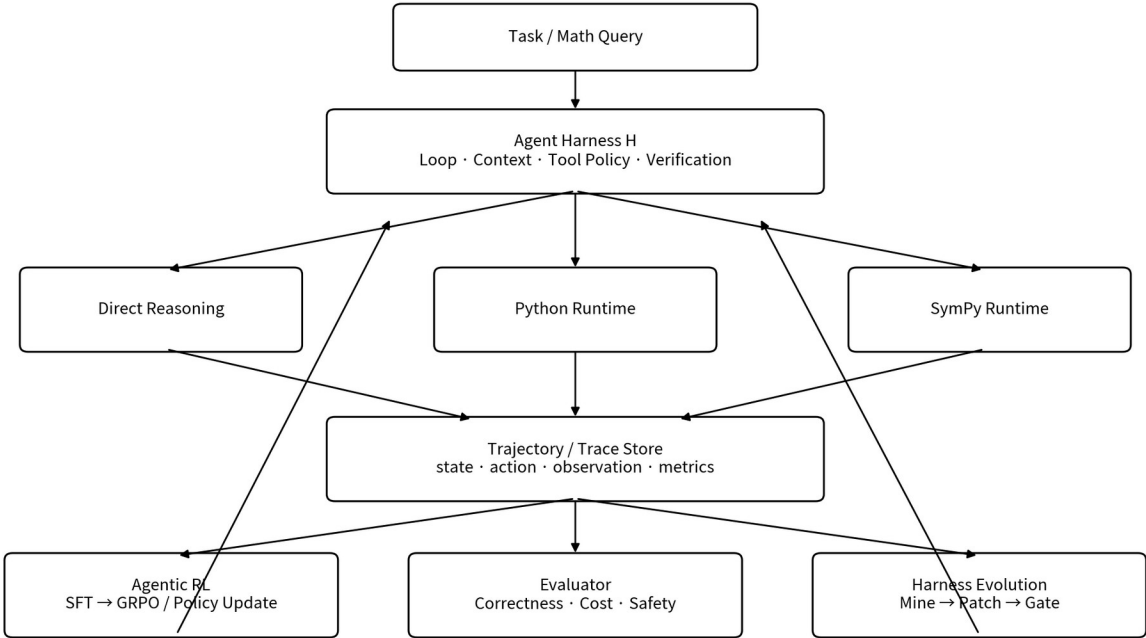


AdaptiveMath-Evo

基于 *Harnessed Agentic RL* 与 *Harness Evolution* 的自进化数学 Agent 技术设计

Technical Design Document • v2.0 • 2026-09

AdaptiveMath-Evo: Model Policy + Harness 双层优化



目标：在现有 Tool-Using Math Agent + Agentic RL 基线上，增加受约束、可验证、可回滚的 Harness Evolution 外环。

0. 执行摘要

项目定位： AdaptiveMath-Evo 是一个面向数学推理任务的 Tool-Using Agent 训练与运行系统。系统同时优化模型策略与模型外执行 Harness，但两条优化链严格解耦：模型参数通过 SFT/Agentic RL 更新，Harness 通过 trajectory-driven search、候选 patch 与 regression gate 更新。

- V1 目标：完成 Constrained Harness Evolution，不允许 LLM 任意修改代码仓库。
- 优化对象：Tool policy、tool preprocessing、retry/termination、verification、context policy 等声明式 Harness 配置。
- 核心闭环：Rollout → Failure Mining → Attribution → Patch Proposal → Quality Gate → Regression Evaluation → Promote/Rollback。
- 关键工程原则：proposal 与 evaluator 解耦；evolution set 与 held-out regression set 隔离；所有 Harness 版本可追踪、可复现、可回滚。
- Agentic RL 与 Harness Evolution 不在同一时间尺度上更新，避免环境/运行时非平稳性导致训练不稳定。

层次	优化变量	主要方法	输出
Cold-start Post-training	θ	SFT / LoRA / QLoRA	初始 Tool-Use Policy
Agentic RL	θ	GRPO / outcome reward / multi-turn rollout	$\theta_t \rightarrow \theta_{t+1}$
Harness Evolution	H	failure mining / constrained patch search / regression gate	$H_t \rightarrow H_{t+1}$
联合系统	(θ, H)	alternating optimization	Adaptive Agent snapshot

1. 问题定义与系统边界

1.1 Agent 形式化

定义 Agent $A = (\pi_\theta, H, E)$ 。 其中 π_θ 表示参数化语言模型策略，H 表示 Agent Harness，E 表示数学任务与工具执行环境。完整 trajectory 记为 $\tau = (s_0, a_0, o_1, \dots, s_T)$ ，其分布满足 $\tau \sim P(\tau \mid \pi_\theta, H, E)$ 。

本项目将 Harness 定义为模型外、直接决定 Agent 执行语义的 runtime/control layer：

```
H = {  
  interaction_loop,  
  tool_schema_and_policy,
```

```
context_policy,
retry_and_termination,
verification_policy,
safety_constraints,
trace_instrumentation
}
```

1.2 非目标 (Non-goals)

- V1 不允许自动生成并执行任意 Python 源码修改；仅支持 schema 约束下的配置/模板 patch。
- V1 不宣称完全自治 self-improving agent；所有候选更新必须经过 deterministic evaluator 与 held-out regression gate。
- V1 不同时持续在线更新 θ 与 H；采用交替阶段化优化。
- V1 不把所有失败归因给 Harness；明确区分 model-capability failure 与 harness-induced failure。

2. 设计依据与技术路线

本方案吸收四类公开工作，但不直接复刻单一框架：

参考方向	可吸收机制	本项目采用方式
ReTool [1]	冷启动 + 多轮实时工具执行 + outcome-driven RL	保留数学工具使用训练主线；扩展 Python + SymPy + Direct 三动作空间
verl-agent / GiGPO [2]	step-independent multi-turn rollout、可自定义 per-step history/memory	训练侧采用环境适配器与分步 trajectory schema，避免全历史简单拼接
Agent Lightning v1.0 [3]	deploy-time harness 拥有 interaction loop；trainer 观察 LLM request-response	架构上分离 Agent Runtime 与 RL Trainer，降低 train-serving mismatch
Self-Harness / HarnessEvolve / HarnessCompass [4–6]	weakness mining、minimal patch、regression gate、credit assignment、component-wise constrained evolution	Evolution 外环采用受约束修改、双 Gate、held-out 验证与版本管理

3. 现有基线与改造点

3.1 现有 Agent 执行基线

现有实现可抽象为：Planner/Router → Tool Caller → Tool Runtime → Critic/Verifier → Executor/Finalizer。工具集合包括 Direct Reasoning、Python 与 SymPy；执行轨迹用于训练与评测。

query

- planner / router
- action $\in \{\text{DIRECT}, \text{PYTHON}, \text{SYMPY}\}$
- tool execution / observation
- critic / verification
- continue | retry | final
- trajectory + reward

3.2 当前结构的主要限制

1. Harness 规则散落在 Python 控制流中，无法作为稳定、可版本化的优化对象。
2. Trajectory 主要服务 RL rollout，缺少面向 root-cause analysis 的统一 error/event schema。
3. Reward 能回答“任务是否成功”，但不能直接回答“失败由 θ 还是 H 引起”。
4. 缺少 candidate harness 的隔离运行、A/B comparison、promotion 与 rollback。
5. 如果直接让 LLM 修改代码，search space 过大且难以保证安全、复现和因果归因。

4. 目标架构

AdaptiveMath-Evo: Model Policy + Harness 双层优化

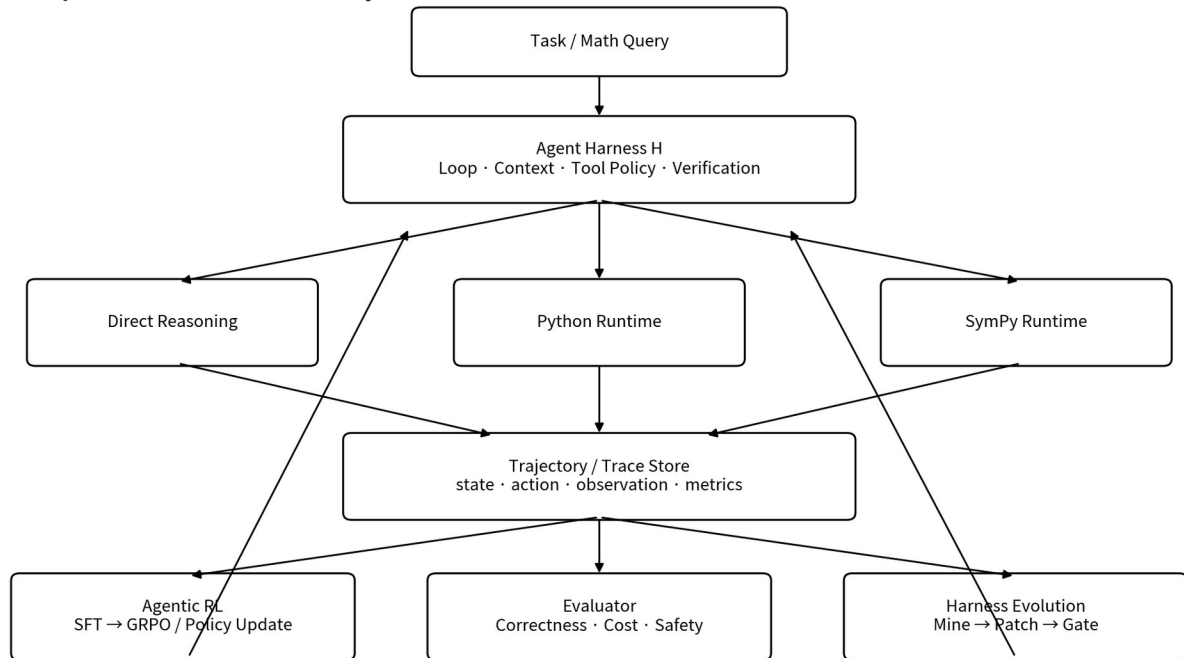
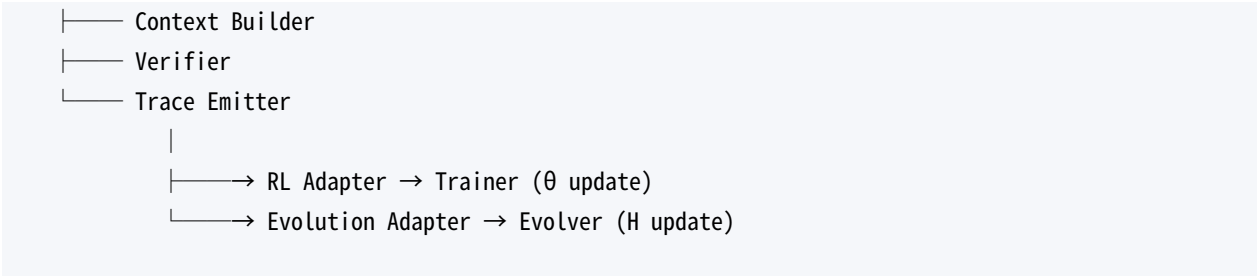


图 1 目标架构。内环 Agentic RL 更新模型策略 θ ；外环 Harness Evolution 更新 H。二者通过统一 trajectory/evaluation layer 共享观测数据，但采用不同 optimizer 与版本周期。

4.1 Runtime / Training 解耦

Agent Runtime (owns interaction loop)

- ├── Harness Config
- ├── Tool Runtime



设计要求：Runtime 不依赖具体 RL 算法；Trainer 不重写 Agent loop。这样可对齐 Harnessed Agentic RL 的工程原则，并确保部署行为与训练 rollout 尽可能一致 [3]。

5. Harness 参数化与可进化搜索空间

核心改造：把可优化 Harness 从硬编码控制流迁移到 typed configuration。Evolution 的最小单元不是“代码 diff”，而是受 schema 约束的 HarnessPatch。

```
harness:
  routing:
    policy_prompt: "..."
    max_tool_switches: 2
  tools:
    python:
      timeout_s: 10
      retry_limit: 1
    sympy:
      normalize_expression: true
      parse_validation: true
      retry_limit: 1
  execution:
    max_steps: 8
    stop_on_verified_answer: true
  verification:
    enabled: true
    numeric_tolerance: 1e-6
    symbolic_equivalence: true
  context:
    max_tool_output_chars: 4000
    retain_last_k_steps: 6
```

组件	V1 是否允许进化	典型 patch	约束
Tool description/schema	是	补充适用边界、参数格式、返回语义	不允许改变工具权限边界
Tool preprocessing	是	LaTeX→SymPy normalization	必须是 allowlist processor

组件	V1 是否允许进化	典型 patch	约束
Retry / timeout	是	retry 1→2	设置全局上限，防止成本膨胀
Termination	是	verified 后提前停止	不能绕过 verifier
Verification trigger	是	数值/符号答案启用二次校验	evaluator 独立于 proposer
Context policy	是	截断/summary/保留最近 K 步	受 token budget 限制
Reward function	否 (V1)	—	防止 evolver 直接改评分规则
任意源代码	否 (V1)	—	避免 unrestricted self-modification

6. Trajectory 数据模型与 Failure Taxonomy

6.1 Trace Schema

```
Trajectory {
  run_id, task_id, harness_version, model_version,
  input, reference_answer,
  steps: [
    {step_id, state_digest, model_action, tool_name,
      tool_args, observation, error_code,
      latency_ms, token_usage}
  ],
  final_answer,
  task_reward,
  verifier_result,
  total_latency_ms,
  total_tokens,
  termination_reason
}
```

每条 trace 必须绑定 harness_version 与 model_version，否则无法进行可复现的 A/B 比较，也无法判断性能变化来自 θ 还是 H 。

6.2 Failure Taxonomy

Failure Type	判定信号	默认归因方向
TOOL_SELECTION_ERROR	选择工具与任务类型明显不匹配，且替代工具可成功	Routing / Tool Policy
TOOL_ARGUMENT_ERROR	schema 校验失败、参数缺失、格式错误	Tool Schema / Prompt
TOOL_EXECUTION_ERROR	timeout/runtime exception	Tool Runtime / Retry
SYMPY_PARSE_ERROR	表达式解析失败	Preprocessor / Tool Contract
PREMATURE_TERMINATION	未验证即结束，或存在未解决子目标	Termination / Verification
LOOP_OR_REDUNDANCY	重复相同 action/observation	Loop / Context / Stop Rule
VERIFICATION_FAILURE	答案可被工具校验但未触发校验	Verification Policy
CAPABILITY_FAILURE	Harness 正常但推理/数学能力不足	Model Policy θ ，不触发 H patch

7. Failure Attribution：决定“改模型还是改 Harness”

Harness Evolution 的核心不是生成 patch，而是 attribution。若 attribution 错误，系统会把模型能力缺陷错误地编码为 prompt/tool hack，导致 shortcut learning、prompt bloat 或 benchmark overfitting。

7.1 V1：规则 + Counterfactual Replay

- 6. Rule-based triage：依据 error_code、tool exception、termination reason 进行第一层组件映射。
- 7. Counterfactual replay：在固定 θ 下，仅替换候选 Harness component 重放失败样本；若结果显著改善，则增强 Harness attribution 置信度。
- 8. Reference-assisted analysis：对有 ground-truth 的数学题，可构造 reference tool path 或 verification path，比较 failed trajectory 的结构差异。
- 9. CAPABILITY_FAILURE 不进入 evolution queue，只进入 RL hard-example pool。

```
AttributionResult {
  failure_type,
  target_component,
  confidence ∈ [0, 1],
  evidence_run_ids[],
  counterfactual_delta,
  eligible_for_harness_patch: bool
}
```

8. Harness Evolution 外环

Harness Evolution 外环：执行、诊断、修改与验证解耦

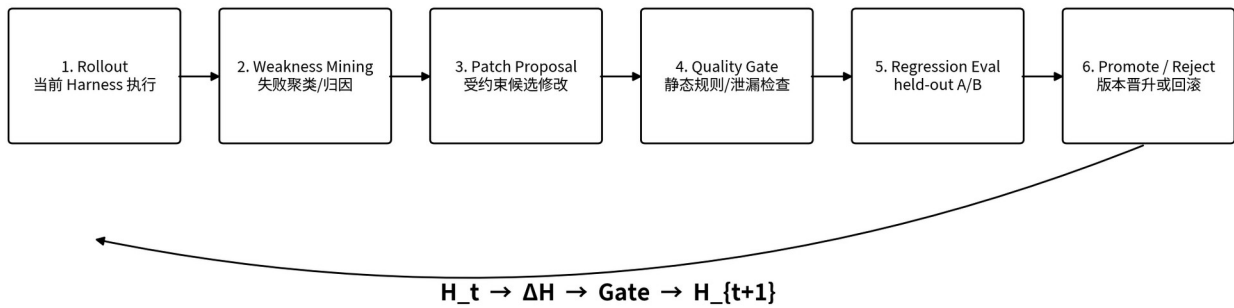


图 2 Evolution pipeline。Proposal 生成与最终裁决分离；候选 patch 必须先过静态质量检查，再通过 held-out regression evaluation。

8.1 Weakness Mining

- 按 failure_type、target_component、task category 聚类，避免单样本驱动修改。
- 设置 minimum_support，例如同类 failure $\geq N$ 才生成 proposal。
- 记录 failure prevalence、expected impact 与受影响任务分布。

8.2 Patch Proposal

```

HarnessPatch {
  patch_id,
  base_version,
  target_component,
  operation,          # update | add | enable | disable
  path,               # e.g. tools.sympy.retry_limit
  old_value,
  new_value,
  evidence_run_ids[],
  expected_effect,
  risk_level
}
  
```

Patch proposer 可以由 LLM 实现，但输出必须通过 JSON Schema/Pydantic 校验；禁止 proposer 直接写入 production config。

8.3 Quality Gate

- Schema validity: 字段、类型、范围、allowlist。
- Data leakage check: 禁止把具体题目答案、测试样本内容注入 Harness。
- Prompt bloat check: 限制 instruction/token 增长。
- Safety invariant: 不得关闭 verifier、sandbox、tool permission boundary。
- Complexity budget: 每轮限制 patch 数量与受影响 component 数量。

8.4 Performance Gate

定义评测向量：

```
M(H) = {
  task_success_rate,
  exact_match_or_symbolic_accuracy,
  tool_success_rate,
  invalid_tool_call_rate,
  avg_steps,
  avg_tokens,
  p95_latency,
  regression_rate
}
```

示例 promotion rule：

```
promote iff:
  Δtask_success_rate >= +1.0 pp
  AND regression_rate <= 2.0%
  AND invalid_tool_call_rate does not increase
  AND avg_tokens <= baseline * 1.10
  AND all safety invariants pass
```

9. 数据切分与评测协议

Harness Evolution 容易对 evolution tasks 过拟合，因此必须把“发现问题的数据”和“证明改进的数据”物理隔离。

Split	用途	是否允许 Evolver 查看	说明
RL Train / Rollout	Agentic RL 采样与 θ 更新	是（通过 trace）	不用于最终 Harness 结论
Evolution Dev	Failure mining + candidate selection	是	可以迭代使用
Regression Set	candidate H_t vs H' 的 A/B gate	仅 evaluator	禁止向 proposer 暴露答案/具体失败模式

Split	用途	是否允许 Evolver 查看	说明
Final Held-out	epoch/里程碑级最终验证	否	只用于最终 snapshot 选择

9.1 必做 Ablation

- Base Agent vs +SFT vs +RL：证明 θ 优化的增益。
- Fixed Harness vs Evolved Harness（固定 θ ）：证明 H 优化的独立增益。
- No Attribution vs Rule Attribution vs Counterfactual Attribution：验证归因模块价值。
- No Gate vs Quality Gate vs Dual Gate：验证回归控制与稳定性。
- 单组件 evolution vs 多组件 evolution：衡量 component interference。

10. 与 Agentic RL 的联合训练策略

如果 θ 与 H 每个 batch 同时更新，则 $P(\tau|\theta,H)$ 的两部分同时变化，造成显著 non-stationarity。V1 采用 alternating optimization，而不是 joint online co-training。

```
Stage A — Cold-start
   $\theta_0 \leftarrow \text{SFT}(\text{tool-augmented trajectories})$ 
   $H_0 \leftarrow \text{human-designed minimal harness}$ 

Stage B — Agentic RL
  freeze  $H_0$ 
   $\theta_1 \leftarrow \text{GRPO}(\text{rollouts under } H_0)$ 

Stage C — Harness Evolution
  freeze  $\theta_1$ 
   $H_1 \leftarrow \text{Evolve}(H_0, \text{traces, regression gate})$ 

Stage D — Re-adaptation (optional)
  freeze  $H_1$ 
   $\theta_2 \leftarrow \text{short RL adaptation under } H_1$ 

Snapshot = ( $\theta_2$ ,  $H_1$ )
```

只有当 Stage B/C/D 均有真实实验结果时，才建议将项目表述升级为 Model–Harness Co-Evolution；否则应准确描述为“Agentic RL + Harness Evolution 双层优化框架”。

11. Agentic RL 训练侧设计

11.1 Action / Observation

Action space:

```
DIRECT(reasoning/final)
PYTHON(code)
SYMPY(expression / operation)
```

Observation:

```
tool stdout / structured result / exception / verifier feedback
```

11.2 Reward

主 reward 必须保持 task outcome 可验证性，工具成本只作为轻量 shaping，避免模型通过“不调用工具”获得成本优势而牺牲正确率。

```
R(τ) = R_answer
      + λ1 * R_tool_validity
      + λ2 * R_verification
      - λ3 * tool_cost
      - λ4 * invalid_or_redundant_actions
```

Constraint: $|\lambda_3|, |\lambda_4| \ll \text{answer correctness weight}$

11.3 训练工程约束

- Tool execution 与 trainer 进程隔离；Python 使用受限 sandbox、timeout、resource quota。
- 训练样本中记录 tool tokens 与 environment tokens 的 mask，明确哪些 token 参与 policy loss。
- 同一 experiment 固定 model checkpoint、Harness version、dataset manifest hash、seed 与 evaluator version。
- 所有 reward computation 可离线复算，便于排查 reward hacking。

12. Harness Registry、版本与回滚

```
harness_registry/
  v0001/
    harness.yaml
    metadata.json
    parent.json
    eval_report.json
  v0002/
    harness.yaml
    patch.json
    metadata.json
```

```
eval_report.json

metadata:
  version
  parent_version
  model_version
  created_at
  evidence_batch
  proposer_version
  evaluator_version
  git_commit
```

- Harness version 必须 immutable；更新通过新版本完成，不覆盖旧版本。
- Promotion 使用 alias (candidate / stable / rollback) 而非修改历史版本。
- 实验结果以 (model_version, harness_version, evaluator_version) 为最小可复现三元组。

13. 建议代码目录

```
adaptive_math_evo/
├── agent/
│   ├── runtime.py
│   ├── loop.py
│   ├── context.py
│   └── verifier.py
├── tools/
│   ├── python_tool.py
│   ├── sympy_tool.py
│   └── preprocessors.py
├── harness/
│   ├── schema.py
│   ├── loader.py
│   ├── registry.py
│   └── configs/
├── traces/
│   ├── schema.py
│   ├── emitter.py
│   └── store.py
├── evolution/
│   ├── miner.py
│   ├── attribution.py
│   ├── proposer.py
│   ├── quality_gate.py
│   ├── regression_runner.py
│   └── promoter.py
```

```

├── rl/
│   ├── env_adapter.py
│   ├── reward.py
│   └── verl_adapter.py
├── evaluation/
│   ├── metrics.py
│   ├── datasets.py
│   └── report.py
└── tests/

```

14. 实施路线（按工程风险递增）

阶段	交付物	验收条件
P0: Harness 参数化	typed config + loader + immutable version	同一版本重复运行行为一致；配置可 hash
P1: Trace/Failure	统一 trace schema + taxonomy + dashboard/report	失败可按 component/task 聚合；RL rollout 不受破坏
P2: Constrained Evolver	miner + proposer + schema gate	能从失败簇生成合法 patch，不能直接改源码
P3: Regression Gate	candidate sandbox + A/B report + promote/reject	自动比较 H_t/H' ；支持 rollback
P4: Attribution Enhancement	counterfactual replay + reference-assisted diagnosis	错误 patch rate 下降；attribution 有可量化指标
P5: RL Re-adaptation	H 更新后短程 RL adaptation	验证 θ/H 交替优化是否有额外收益

15. 测试与验收标准

15.1 单元测试

- Harness schema 范围校验、默认值、序列化与 hash 稳定性。
- Patch apply 必须 deterministic，并能生成 inverse patch。
- Failure taxonomy 对已知 fixtures 输出稳定分类。
- Quality gate 能拦截答案泄漏、超限 prompt、非法 processor 与 verifier disable。

15.2 集成测试

- 固定 $\theta + H_0/H_1$ 对同一 regression set 并行运行，报告完整指标与逐样本 diff。
- candidate 崩溃/timeout 时自动判 reject，不影响 stable harness。

- promotion 后可一键 rollback 到 parent_version。
- 训练进程只通过公开接口读取 Harness，不允许直接依赖 evolution 模块内部状态。

16. 最终应产出的实验表

Experiment	Success/Acc	Tool Error	Avg Steps	Tokens	Regression
Base Model + H0	—	—	—	—	—
SFT + H0	—	—	—	—	—
Agentic RL + H0	—	—	—	—	—
Agentic RL + H1 (Evolved)	—	—	—	—	—
Agentic RL Re-adapted + H1	—	—	—	—	—

注意：所有简历量化结果应来自这张表或其可复现实验记录。没有跑出的指标保留为空，不使用估计值。

17. 技术表述边界（避免过度包装）

完成程度	可以使用的表述	暂不建议表述
完成 P0–P3	“实现 trajectory-driven constrained Harness Evolution，支持 failure mining、候选 patch、回归 gate 与版本回滚”	“通用自进化 Agent”
完成 P4	“引入 counterfactual/reference-assisted attribution 降低错误 Harness 更新”	“解决了长期 credit assignment”
完成 P5 且有消融	“构建 Agentic RL 与 Harness Evolution 交替优化流程”	“Model–Harness Co-Evolution” 若无联合实验
仅有设计未实现	“设计/规划”	“实现/落地/提升 X%”

18. 关键风险与缓解

风险	表现	缓解方案
Benchmark overfitting	Harness 记住 evolution tasks	dev/regression/final held-out 隔离；泄漏扫描
Prompt bloat	每轮追加规则，token 成本持续增长	minimal patch；instruction length budget；定期 consolidation
Credit assignment error	把模型能力问题误判为 Harness 问题	CAPABILITY_FAILURE 分类；counterfactual replay
Catastrophic regression	新任务提升，旧能力下降	recent-batch + global regression gate；版本 rollback
Non-stationarity	θ /H 同时变化导致训练不稳定	alternating optimization；固定 snapshot
Reward hacking	Agent 利用 evaluator 漏洞	reward 离线复算；独立 verifier；异常轨迹审计

19. MVP Definition of Done

- Harness 已从主要硬编码规则中抽离为 typed config，并具有版本 hash。
- 每次 rollout 自动产出结构化 trajectory，包含 model/harness/evaluator 版本。
- 至少覆盖 5 类 failure，并能将 CAPABILITY_FAILURE 与 harness-induced failure 分离。
- Evolver 能对至少 3 类 Harness component 生成受约束 patch。
- 候选 patch 必须经过 Quality Gate 和独立 Regression Gate。
- 能生成 H_t vs $H_{candidate}$ 的逐指标评测报告并自动 promote/reject。
- 所有 promoted Harness 可回滚；历史版本不可变。
- 至少完成一次固定 θ 的 Harness ablation，证明提升不是模型参数变化导致。

20. 参考文献与实现参考

- [1] Feng, J. et al. “ReTool: Reinforcement Learning for Strategic Tool Use in LLMs.” ICLR 2026; arXiv:2504.11536.
- [2] langfengQ et al. “verl-agent: RL training for LLM/VLM agents; Group-in-Group Policy Optimization.” NeurIPS 2025. Repository: github.com/langfengQ/verl-agent.
- [3] He, Z. et al. “Agent Lightning v1.0: Towards Harnessed Agentic RL.” Microsoft Research, Aug 2026.
- [4] Zhang, H. et al. “Self-Harness: Harnesses That Improve Themselves.” arXiv:2606.09498, Jun 2026.
- [5] Jiang, W. et al. “HarnessEvolve: Learning from Reference Trajectories for Reliable Agent Self-Evolution.” arXiv:2609.00829, Sep 2026.
- [6] Zhang, L. et al. “HarnessCompass: Guiding Automatic Harness Evolution toward Generalizable and Effective Agent Harnesses.” arXiv:2608.01918, Aug 2026.